Exercises: Branching

Introduction to computer programming

Management and Artificial Intelligence

Recap examples

Simple `if` statement

```
x = 1

# If x is greater than 5, this block will be executed
if x > 5:
    print("x is greater than 5")
```

Explanation:
The `if` statement checks if the condition (`x > 5`) is `True`. Since `x` is 10, the condition is true, so the print statement is executed.

`if-else` statement

```
x = 3

# If x is greater than 5, this block will be executed, else the other block
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

```
x is not greater than 5
```

Explanation:
The `else` block is executed if the `if` condition is `False`. Since `x = 3`, the condition is false, so the `else` block runs.

`if-elif-else` statement

```
x = 6

# Checking multiple conditions using if, elif, and else
if x > 3:
    print("x is greater than 3")
elif x > 5:
    print("x is greater to 5")
else:
    print("x is less than 5")
```

```
x is greater than 3
```

Explanation:
`elif` allows for checking multiple conditions. Here, the program checks if `x` is greater than 5, equal to 5, or less than 5. Since `x == 5`, the corresponding block is executed.

Nested `if` statements

```
x = 10
y = 5

# Nested branching
if x > 5:
    if y > 5:
        print("Both x and y are greater than 5")
    else:
        print("x is greater than 5, but y is not")
else:
    print("x is not greater then 5, we don't know about y")
```

```
x is greater than 5, but y is not
```

Explanation:
You can nest `if` statements within each other. In this case, the program first checks if `x > 5`, then checks if `y > 5`. Since `x > 5` and `y <= 5`, the nested `else` block is executed.

Using `and` in conditions

```
x = 10
y = 8

# Using and to check multiple conditions
if x > 5 and y > 5:
    print("Both x and y are greater than 5")
else:
    print("At least one of x or y is not greater than 5")
```

```
Both x and y are greater than 5
```

Explanation:
The `and` operator ensures that both conditions (`x > 5` and `y > 5`) must be `True` for the code block to run. Since both conditions are true, the first print statement is executed.

Using `or` in conditions

```
x = 4
y = 10

# Using or to check multiple conditions
if x > 5 or y > 5:
    print("At least one of x or y is greater than 5")
else:
    print("Neither x nor y is greater than 5")
```

```
At least one of x or y is greater than 5
```

Explanation:
The `or` operator ensures that if either condition is `True`, the code block will run. Since `y > 5`, the first block is executed even though `x <= 5`.

Using `not` in conditions

```
x = 4

# Using not to negate the condition
if not x > 5:
    print("x is not greater than 5")
```

```
x is not greater than 5
```

Explanation:
The `not` operator negates the condition. Here, the condition `x > 5` is false, but `not x > 5` is true, so the print statement is executed.

Checking for membership in a list

```
fruits = "applebananacherryapple"

# Checking if an element is in the list
if "apple" in fruits:
    print("Apple is in the string of fruits")
else:
    print("Apple is not in the string")
```

```
Apple is in the string of fruits
```

Explanation:
The `in` operator checks if an element exists within a collection (like a list). Since "apple" is in `fruits`, the first print statement is executed.

Ternary (conditional) operator

```
x = 7

if x > 5:
    message = "x is greater than 5"
else:
    message = "x is not greater than 5"
print(message)

# Short form of if-else using ternary operator
message = "x is greater than 5" if x > 5 else "x is not greater than 5"
print(message)
```

```
x is greater than 5
x is greater than 5
```

Explanation:
The ternary operator allows for a shorter syntax for `if-else` statements. It evaluates `x > 5` and assigns the result to `message` accordingly. Since `x > 5`, the string "x is greater than 5" is assigned to `message`.

```
x = 5

if x > 7:
    msg = "A"
else:
    msg = "B"
print(msg)

msg = "A" if x > 7 else "B"
print(msg)
```

```
B
B
```

Explanation:
The ternary operator allows for a shorter syntax for `if-else` statements. It evaluates `x > 7` and assigns the result to `msg` accordingly. Since `x > 7`, the string "B" is assigned to `msg`.

Using `pass` for empty branches

```
x = 10

# Pass is used as a placeholder when you need a block but don't want to do anything
if x > 5:
    pass  # Later logic can be added here
else:
    print("x is not greater than 5")

print(x)
```

```
10
```

Explanation:
The `pass` statement is a placeholder when you need to write an empty block of code. This is useful when you plan to implement functionality later but don't want an error for an empty block.

Exercises

- Notebook with exercises on branching (no hints)
- Notebook with exercises on branching (with pseudocode)
- Notebook with exercises on branching (with solutions)